Memory-bound workloads in gaming

Where Lavik fits, sizing estimates, and a benchmark plan for the use-case whitepaper

Two patterns: leaderboards are hot, player state is big

Leaderboards

Sorted sets of player scores

- Small to medium in size
- Read and updated constantly
- Limited by CPU and latency
- **Lavik fit: supporting case**

Player state

Profiles, inventory, sessions, tokens, transaction cache

- Hundreds of MB to several TB
- Most of it is cold at any moment
- Limited by memory cost
- **Lavik fit: lead case**

Leaderboard sizing estimates

Leaderboard type	Sorted sets per cluster	Members each	Memory each	Notes
Global or season	1 to 20	1M to 100M	100 MB to 10 GB	Long-lived; top-N and rank lookups
Per match or round	10K to 1M live	10 to 100	A few KB	Short TTL; high create and expire churn
Guild, friend or event	100K to 10M	100 to 1,000	10 to 100 KB	Many mid-size sets; mixed hot and cold

Rule of thumb: about 100 bytes per member with short player IDs. Small sets (under about 128 members) use a compact encoding and cost far less per member.

Player state is where memory cost hurts

Sessions and tokens

100 MB to 50 GB

Hot, short-lived; less of a Lavik case

Player profiles

50 GB to 2 TB

Large, mostly cold; strong Lavik case

Inventory and items

50 GB to 2 TB

Large, mostly cold; strong Lavik case

Transaction cache

10 GB to 500 GB

Mixed; durability matters

At these sizes, an all-in-memory design means buying servers for RAM, not compute. That is the cost Lavik removes.

Public case studies show both pains at scale

MyTeam11

Fantasy sports, India

15M+

users

25K+

ops per second, set to double

Leaderboards, sessions and authentication in one in-memory store. Load spikes in the 30 minutes before each cricket match.

Etermax

Trivia Crack, Word Crack

25M

users at peak

~3.9 TB

RAM: 16 endpoints of 244 GB

Hit a memory wall and CPU limits (up to 95% CPU). Single-threaded design could not use all cores on large instances.

Benchmark plan for the whitepaper

Workload	Shape	Mix	Measure
A. Global leaderboard	10 sorted sets x 10M members	80% reads (top-100, rank), 20% score updates	QPS, p99 and p99.9 latency
B. Match leaderboards	1M sorted sets x 50 members, 30-min TTL	Create, update, expire churn	Throughput under churn, memory
C. Player state (lead)	100M hashes x 2 KB (~200 GB), 5 to 10% hot	90% reads, 10% writes	Cost per TB, nodes needed, p99 on cold reads

Compare each against Redis and Valkey on equal hardware, with persistence settings stated, so the results stand up to a buyer's architect.

Next steps

1	**Engineering**	Run workloads A, B and C; share raw results and setup
2	**Validation**	Check sizing ranges with two or three gaming operators
3	**Whitepaper**	Lead with player state; leaderboards as a supporting case
4	**Pilot**	Offer a free pilot to one gaming design partner